Unit II

2. Software Development Life Cycle(SDLC)and Methodologies

2.1 Introduction
2.2 Activities of SDLC
2.3 A Generic Process Model
2.4 Prescriptive Process models
2.4.1 Waterfall Model
2.4.2 Incremental Process Models
2.4.3 Evolutionary process Models (Prototyping and Spiral Model)
2.5 Concurrent Models, Types

2.1 Introduction:

Introduction to SDLC and Methodologies:

The **Software Development Life Cycle (SDLC)** is a systematic process used by software developers to design, develop, test, and deploy software efficiently and effectively. It ensures high-quality software is delivered on time and within budget. The main phases of SDLC include **requirement analysis, planning, design, development, testing, deployment, and maintenance**.

There are various **SDLC methodologies** that guide how these phases are carried out. Common methodologies include:

Waterfall Model – A linear and sequential approach.
Agile Model – An iterative, flexible method with continuous feedback.
Spiral Model – Combines iterative development with risk analysis.
V-Model – An extension of Waterfall with a focus on validation and verification.

Each methodology suits different types of projects based on complexity, time, and customer requirements.

2.2 Activities of SDLC:

The Software Development Life Cycle (SDLC) includes several key activities to ensure the successful creation and maintenance of software. These are:

1. Requirement Gathering and Analysis:

In this phase, stakeholders' needs are collected and analyzed to understand what the software must do.

2. System Design:

Based on the requirements, the system's architecture, data flow, and interface designs are prepared.

3. Implementation (Coding):

Developers write code to build the software using suitable programming languages and tools.

4. Testing:

The software is tested to identify and fix bugs, and to ensure it meets the required quality standards.

5. Deployment:

The software is released to the users and installed in the production environment.

6. Maintenance:

Ongoing updates, bug fixes, and improvements are made after deployment to keep the software running smoothly.

2.3 A Generic Process Model:

A **Generic Process Model** describes the basic framework used to build software systematically. It outlines common activities that are found in most software development processes, regardless of the specific methodology used. The key components are:

1. Communication:

Involves interaction with stakeholders to gather requirements and understand the project's goals.

2. Planning:

Estimating time, cost, resources, and tasks required to develop the software. A detailed project plan is created.

3. Modeling:

Creating software models such as data flow diagrams, architecture, and design to represent the system structure.

4. Construction:

This includes coding (implementation) and testing. Developers build the actual software and test it for errors.

5. Deployment:

The final product is delivered to users. It may include installation, training, and ongoing support or maintenance.

This model provides a universal structure that can be adapted by various development methodologies like Agile, Waterfall, etc.

2.4.1 Waterfall Model:

The **Waterfall Model** is a **linear and sequential software development model** where each phase must be completed before moving to the next. It is one of the earliest and simplest SDLC models.

Phases of the Waterfall Model:

1. Requirement Analysis:

All software requirements are gathered from the client and documented clearly.

2. System Design:

The overall system architecture and design specifications are prepared based on the requirements.

3. Implementation (Coding):

Developers write the actual code according to the design specifications.

4. Testing:

The developed software is tested to find and fix bugs or errors.

5. Deployment:

The tested software is deployed to the user's environment.

6. Maintenance:

After deployment, any errors or updates are handled during the maintenance phase.

Key Features:

Simple and easy to use
Best suited for small, well-defined projects
Difficult to go back to a previous phase once completed

The Waterfall Model works well when requirements are clear from the beginning and unlikely to change.

2.4.2 Incremental Process Models:

The **Incremental Process Model** is a **software development approach** where the system is developed and delivered in small, manageable parts called **increments**. Each increment adds new functionality to the existing system.

Key Features:

1. Divided Development:

The complete system is broken into smaller components that are developed and delivered step-by-step.

2. Partial Implementation:

A working version of the software is delivered early, and new features are added in later increments.

3. User Feedback:

Users can provide feedback after each increment, allowing for changes and improvements in the next stages.

4. Testing in Phases:

Each increment is tested individually, and then integrated with the previous version.

5. Reduced Risk:

Since only a portion of the software is developed at a time, risks can be identified and

resolved early.

Example:

An online shopping app may first release a basic version with product browsing, and later add cart, payment, and tracking features in future increments.

This model is suitable for **medium to large projects** where requirements are expected to evolve over time.

2.4.3 Evolutionary process Models (Prototyping and Spiral Model):

Evolutionary models focus on **developing the software in small steps**, refining it through multiple versions until the final system is complete. Two main types are **Prototyping** and **Spiral Model**.

1. Prototyping Model:

A **prototype** (working model) of the software is built quickly to understand user requirements.

Users interact with the prototype and give feedback.

Based on feedback, the prototype is improved until final requirements are clear.

Then, the actual system is developed.

Advantage: Helps clarify vague requirements and improves user satisfaction.

2. Spiral Model:

Combines features of **Waterfall** and **Prototyping** with a strong emphasis on **risk analysis**.

The process is divided into **spirals (iterations)**, each containing:

1. **Planning**
2. **Risk analysis**
3. **Engineering (development & testing)**
4. **Evaluation**

Each spiral results in a more complete version of the software.

Advantage: Suitable for large, high-risk projects; manages risks early.

These models support **flexibility** and **customer involvement**, making them ideal for complex and evolving projects.

2.5 Concurrent Models, Types:

The **Concurrent Process Model** is a software development approach where **different activities (phases)** such as design, coding, and testing **can occur simultaneously**, rather than sequentially.

It is ideal for **complex and real-time systems** where various development tasks need to progress in parallel.

Key Features:

All SDLC activities are represented as **states** (e.g., waiting, active, completed).

Multiple activities can be in progress at the same time.

It provides a **realistic representation** of a project's progress, especially when changes are frequent.

Supports **incremental and evolutionary development**.

Types of Concurrent Models:

1. **Event-Driven Model:**

   Activities start based on specific events (e.g., requirement approval triggers design).

2. **State-Transition Model:**

   Each activity is treated as a state, and transitions occur when conditions are met (e.g., coding starts when design is approved).

Use Case:

Often used in **client-server systems**, **real-time systems**, or **Agile environments** where continuous changes and parallel workflows are common.

The Concurrent Model provides **flexibility** and better **project control**, especially for **dynamic and large scale projects**.